Document number: P0645R4
Audience: Library Evolution, Library
Author: Victor Zverovich

2018-11-25

Text Formatting

Revision History

Changes since R3

- Use `OutputIterator` concept in formatting functions that take output iterators and replace the `Size` template parameter with `iter_difference_t`.
- Rename `*parse_context` to `*format_parse_context`.
- Disallow consecutive zeros in `arg-id` and `width`.
- Specify formatting of arithmetic and pointer types in terms of `to_chars`.
- Clarify what is the current locale.
- Use the shortest round-trip format as the default floating-point formatting.

- Add missing specializations of `formatter`.
- Clarify that the `sign` option applies to floating-point infinity and NaN.
- Propose adding a new entry `<format>` to table 18 in section 15.5.1.2 [headers].
- Add section numbers in Proposed wording.
- Add a feature test macro.
- Remove tautological Ensures clauses.
- Merge two `make_format_args` overloads.
- Replace `args()` with `arg(size_t)` in `basic_format_context`.
- Make `format_arg_store` exposition-only.

Changes since R2

- Add section Argument visitation with an example of how to use the visitation API to implement dynamic format specifiers.
- Rename `visit` to `visit_format_arg` to distinguish from `variant`'s `visit`.
- Merge [format.syntax] into [format.functions].
- Remove the restriction that `'\0'` cannot be used as a fill character.
- Replace Postconditions with Ensures.

Changes since R1

- Add the `format_to_n` function taking an output iterator and a size.
- Add a note that compile-time processing of format strings applies to user-defined types with custom parsers to section Compile-time processing of format strings.
- Rename `count` to `formatted_size`.
- Drop nested namespace `fmt`.
- Add `format` prefix or infix to class and function names to avoid potential name collision after removing the nested namespace.
- Improve wording.
- Replace the requirement of implementing a formatter via ostream insertion operator if the latter is provided with a note that it can be implemented in such way.
- Expand the Acknowledgements section and remove "Reply to".

Changes since R0

- Add section Compile-time processing of format strings.
- Separate parsing and formatting in the extension API replacing `format_value` function template with class template `formatter` to allow compile-time processing of format strings.
- Change return type of `format_to` and `vformat_to` to `OutputIterator` in synopsis.
- Remove sections Null-terminated string view and Format string, and replace `basic_cstring_view` with `basic_string_view`.
- Add a link to the implementation in Introduction.
- Add a note regarding time formatting and compatibility with D0355 "Extending <chrono> to Calendars and Time Zones" [16] to section Extensibility.
- Rename `basic_args` to `basic_format_args`.
- Rename `is_numeric` to `is_arithmetic`.
- Add the `count` function that counts the number of characters and use it to define output ranges.
- Remove `basic_buffer` and section Formatting buffer and replace buffers with output iterators.
- Add Appendix A: Benchmarks.
- Explain the purpose of the type-erased API in more details in the Binary footprint section.
- Add Appendix B: Binary code comparison.
- Add formatting function overloads for `wchar_t` strings.

Introduction

Even with proliferation of graphical and voice user interfaces, text remains one of the main ways for humans to interact with computer programs and programming languages provide a variety of methods to perform text formatting. The first thing we do when learning a new programming language is often write a "Hello, World!" program that performs simple formatted output.

C++ has not one but two standard APIs for producing formatted output, the `printf` family of functions inherited from C and the I/O streams library (iostreams). While iostreams are usually the recommended

way of producing formatted output in C++ for safety and extensibility reasons, `printf` offers some advantages, such as an arguably more natural function call API, the separation of formatted message and arguments, possibly with argument reordering as a POSIX extension, and often more compact source and binary code.

This paper proposes a new text formatting library that can be used as a safe and extensible alternative to the `printf` family of functions. It is intended to complement the existing C++ I/O streams library and reuse some of its infrastructure such as overloaded insertion operators for user-defined types.

Example:

```
string message = format("The answer is {}.", 42);
```

A full implementation of this proposal is available at https://github.com/fmtlib/fmt/tree/std.

Design

Format string syntax

Variations of the `printf` format string syntax are arguably the most popular among the programming languages and C++ itself inherits `printf` from C [1]. The advantage of the `printf` syntax is that many programmers are familiar with it. However, in its current form it has a number of issues:

- Many format specifiers like `hh`, `h`, `l`, `j`, etc. are used only to convey type information. They are redundant in type-safe formatting and would unnecessarily complicate specification and parsing.
- There is no standard way to extend the syntax for user-defined types.
- There are subtle differences between different implementations. For example, POSIX positional arguments [2] are not supported on some systems [6].
- Using `'%'` in a custom format specifier poses difficulties, e.g. for `put_time`-like time formatting.

Although it is possible to address these issues while maintaining resemblance to the original `printf` format, this will still break compatibility and can potentially be more confusing to users than introducing a different syntax.

Therefore we propose a new syntax based on the ones used in Python [3], the .NET family of languages [4], and Rust [5]. This syntax employs `'{'` and `'}'` as replacement field delimiters instead of `'%'` and it is described in detail in [format.functions]. Some advantages of the proposal are:

- A consistent and easy to parse mini-language focused on formatting rather than conveying type information
- Extensibility and support for custom format strings for user-defined types
- Positional arguments
- Support for both locale-specific and locale-independent formatting (see Locale support)
- Formatting improvements such as better alignment control, fill character, and binary representation

The syntax is expressive enough to enable translation, possibly automated, of most `printf` format strings. The correspondence between `printf` and the new syntax is given in the following table:

printf	**new**
-	<
+	+
space	*space*
#	#
0	0
hh	unused
h	unused
l	unused
ll	unused
j	unused
z	unused
t	unused
L	unused

c	c (optional)
s	s (optional)
d	d (optional)
i	d (optional)
o	o
x	x
X	X
u	d (optional)
f	f
F	F
e	e
E	E
a	a
A	A
g	g (optional)
G	G
n	unused
p	p (optional)

Width and precision are represented similarly in `printf` and the proposed syntax with the only difference that runtime value is specified by `'*'` in the former and `'{}'` in the latter, possibly with the index of the argument inside the braces:

```
printf("%*s", 10, "foo");
format("{:{}}", "foo", 10);
```

As can be seen from the table above, most of the specifiers remain the same which simplifies migration from `printf`. A notable difference is in the alignment specification. The proposed syntax allows left, center, and right alignment represented by `'<'`, `'^'`, and `'>'` respectively which is more expressive than the corresponding `printf` syntax. The latter only supports left and right alignment.

The following example uses center alignment and `'*'` as a fill character:

```
format("{:*^30}", "centered");
```

resulting in `"**********centered**********"`. The same formatting cannot be easily achieved with `printf`.

In addition to positional arguments, the grammar can be easily extended to support named arguments.

Extensibility

Both the format string syntax and the API are designed with extensibility in mind. The mini-language can be extended for user-defined types and users can provide functions that implement parsing, possibly at compile time, and formatting for such types.

The general syntax of a replacement field in a format string is

```
replacement-field ::= '{' [arg-id] [':' format-spec] '}'
```

where `format-spec` is predefined for built-in types, but can be customized for user-defined types. For example, the syntax can be extended for `put_time`-like date and time formatting

```
time_t t = time(nullptr);
string date = format("The date is {0:%Y-%m-%d}.", *localtime(&t));
```

by providing a specialization of `formatter` for `tm`:

```
template<>
  struct formatter<tm> {
    constexpr format_parse_context::iterator parse(format_parse_context& ctx);

    template<class FormatContext>
      typename FormatContext::iterator format(const tm& tm, FormatContext& ctx);
  };
```

The `formatter<tm>::parse` function parses the `format-spec` portion of the format string corresponding to

the current argument and `formatter<tm>::format` formats the value and writes the output via the iterator `ctx.begin()`.

Note that date and time formatting is not covered by this proposal but formatting facilities provided by D0355 "Extending <chrono> to Calendars and Time Zones" [16] can be easily implemented using this extension API.

An implementation of `formatter<T>::format` can use ostream insertion `operator<<` for user-defined type `T` if available.

The extension API is based on specialization instead of the argument-dependent lookup (ADL), because the `parse` function doesn't take the object to be formatted as an argument and therefore some other way of parameterizing it on the argument type `T` such as introducing a dummy argument has to be used, e.g.

```
constexpr auto parse(type<T>, format_parse_context& ctx);
```

Another problem with ADL-based approach is compile-time performance as pointed out in [20]:

> Overload resolution on `operator<<` tends to get expensive for larger projects with hundreds or thousands of candidates in the overload set. This seems hard to resolve, since choosing a different name for `operator<<` simply shifts the expense of overload resolution to a differently-named function.

Safety

Formatting functions rely on variadic templates instead of the mechanism provided by `<cstdarg>`. The type information is captured automatically and passed to formatters guaranteeing type safety and making many of the `printf` specifiers redundant (see Format String Syntax). Memory management is automatic to prevent buffer overflow errors common to `printf`.

Locale support

As pointed out in P0067 "Elementary string conversions"[17] there is a number of use cases that do not require internationalization support, but do require high throughput when produced by a server. These include various text-based interchange formats such as JSON or XML. The need for locale-independent functions for conversions between integers and strings and between floating-point numbers and strings has also been highlighted in [20]. Therefore a user should be able to easily control whether to use locales or not during formatting.

We follow Python's approach [3] and designate a separate format specifier `'n'` for locale-aware numeric formatting. It applies to all integral and floating-point types. All other specifiers produce output unaffected by locale settings. This can also have positive effect on performance because locale-independent formatting can be implemented more efficiently.

Positional arguments

An important feature for localization is the ability to rearrange formatting arguments as the word order may vary in different languages [7]. For example:

```
printf("String `%s' has %d characters\n", string, length(string)));
```

A possible German translation of the format string might be:

```
"%2$d Zeichen lang ist die Zeichenkette `%1$s'\n"
```

using POSIX positional arguments [2]. Unfortunately these positional specifiers are not portable [6]. The C++ I/O streams don't support such rearranging of arguments by design because they are interleaved with the portions of the literal string:

```
cout << "String `" << string << "' has " << length(string) << " characters\n";
```

The current proposal allows both positional and automatically numbered arguments, for example:

```
format("String `{}' has {} characters\n", string, length(string)));
```

with the German translation of the format string being:

```
"{1} Zeichen lang ist die Zeichenkette `{0}'\n"
```

Performance

The formatting library has been designed with performance in mind. It tries to minimize the number of virtual function calls and dynamic memory allocations done per formatting operation. In particular, if formatting output can fit into a fixed-size array allocated on stack, it should be possible to avoid both of them altogether by using a suitable API.

The `format_to` function takes an arbitrary output iterator and, for performance reasons, can be specialized for random-access and contiguous iterators as shown in the reference implementation [14].

The locale-independent formatting can also be implemented more efficiently than the locale-aware one. However, the main goal for the former is to support specific use cases (see Locale support) rather than to improve performance.

See Appendix A: Benchmarks for a small performance comparison of the reference implementation of this proposal versus the standard formatting facilities.

Binary footprint

In order to minimize binary code size each formatting function that uses variadic templates can be implemented as a small inline wrapper around its non-variadic counterpart. This wrapper creates a `basic_format_args` object, representing an array of type-erased argument references, with `make_format_args` and calls the non-variadic function to do the actual work. For example, the `format` variadic function calls `vformat`:

```
string vformat(string_view fmt, format_args args);

template<class... Args>
  inline string format(string_view fmt, const Args&... args) {
    return vformat(fmt, make_format_args(args...));
  }
```

`basic_format_args` can be implemented as an array of tagged unions. If the number of arguments is small then the tags that indicate the arguments types can be combined and passed into a formatting function as a single integer. This single integer representing all argument types is computed at compile time and only needs to be stored, resulting in smaller binary code.

Given a reasonable optimizing compiler, this will result in a compact per-call binary code, effectively consisting of placing argument pointers (or, possibly, copies for primitive types) and packed tags on the stack and calling a formatting function. See Appendix B: Binary code comparison for a specific example.

Exposing the type-erased API rather than making it an implementation detail and only providing variadic functions allows applying the same technique to the user code. For example, consider a simple logging function that writes a formatted error message to `clog`:

```
template<class... Args>
  void log_error(int code, string_view fmt, const Args&... args) {
    clog << "Error " << code << ": " << format(fmt, args...);
  }
```

The code for this function will be generated for every combination of argument types which can be undesirable. However, if we use the type-erased API, there will be only one instance of the logging function while the wrapper function can be trivially inlined:

```
void vlog_error(int code, string_view fmt, format_args args) {
  clog << "Error " << code << ": " << vformat(fmt, args);
}

template<class... Args>
  inline void log_error(int code, string_view fmt, const Args&... args) {
    vlog_error(code, fmt, make_format_args(args...));
  }
```

The current design allows users to easily switch between the two approaches.

For similar reasons formatting functions take format strings as instances of `basic_string_view` instead of being parameterized on a string type. This allows implicit conversions from null-terminated character strings and `basic_string` without affecting code size.

Format strings and `char_traits`

Format strings intentionally use the default character traits in the formatting functions API because making traits customizable creates potential problems with handling format specifications.

Consider the following example:

```
using ci_string_view = std::basic_string_view<char, ci_char_traits>;
auto s = format(ci_string_view("{:X}"), 0xCAFE);
```

where `ci_char_traits` is a character trait with case-insensitive comparison.

Since the format specification grammar is case-sensitive, the behavior of such code is no longer intuitive. There are several options:

1. ill-formed (current proposal),
2. well-formed and `s == "cafe"`,
3. well-formed and `s == "CAFE"`.

Option 2 is a potential pessimization because comparisons will have to go through traits while option 3 effectively ignores traits.

Existing standard functions that take format strings (`printf`, `put_time`, `strftime`) do not allow passing character traits.

Compile-time processing of format strings

It is possible to parse format strings at compile time with `constexpr` functions which has been demonstrated in the reference implementation [14] and in [18]. Unfortunately a satisfactory API cannot be provided using existing C++17 features. Ideally we would like to use a function call API similar to the one proposed in this paper:

```
template<class String, class... Args>
  string format(String fmt, const Args&... args);
```

where `String` is some type representing a compile-time format string, for example

```
struct MyString {
  static constexpr string_view value() { return string_view("{}", 2); }
};
```

However, requiring a user to create a new type either manually or via a macro for every format string is not acceptable. P0424R1 "Reconsidering literal operator templates for strings" [15] provides a solution to this problem based on user-defined literal operators. If this or other similar proposal for compile-time strings is accepted into the standard, it should be easy to provide additional formatting APIs that make use of this feature. Obviously runtime checks will still be needed in cases where the format string is not known at compile time, but as shown in Appendix A: Benchmarks even with runtime parsing performance can be on par with or better than that of existing methods.

Compile-time processing of format strings can work with a user-defined type `T` if the `formatter<T>::parse` function is `constexpr`.

Argument visitation

The argument visitation API can be used to implement dynamic format specifiers, for example, width and precision passed as additional formatting arguments as opposed to being encoded in the format string itself:

```
int width = 10;
int precision = 3;
auto s = format("{0:{1}.{2}f}", 12.345678, width, precision);
// s == "    12.346"
```

An example of a user-defined formatter with dynamic width:

```
struct Answer {};

template<>
  struct formatter<Answer> {
    int width_arg_index = 0;
```

```
    // Parses dynamic width in the format "{<digit>}".
    auto parse(format_parse_context& parse_ctx) {
      auto iter = parse_ctx.begin();
      auto get_char = [&]() { return iter != parse_ctx.end() ? *iter : 0; };

      if (get_char() != '{')
        return iter;
      ++iter;
      char c = get_char();
      if (!std::isdigit(c) || (++iter, get_char()) != '}')
        throw format_error("invalid format");
      width_arg_index = c - '0';
      return ++iter;
    }

    auto format(Answer, format_context& format_ctx) {
      auto arg = format_ctx.args().get(width_arg_index);
      int width = visit_format_arg([](auto value) -> int {
        if constexpr (!std::is_integral_v<decltype(value)>)
          throw format_error("width is not integral");
        else if (value < 0 || value > std::numeric_limits<int>::max())
          throw format_error("invalid width");
        else
          return value;
        }, arg);
      return format_to(format_ctx.out(), "{:{}}", 42, width);
    }
  };

std::string s = format("{0:{1}}", Answer(), 10);
// s == "        42"
```

In many cases formatting can be delegated to standard formatters which makes manual handling of dynamic specifiers unneccessary, but the latter is still important for more complex cases.

This API can also be used to implement a custom formatting engine, such as the one compatible with the `printf` syntax, to provide some of the benefits of the current proposal to the legacy code.

Impact on existing code

The proposed formatting API is defined in the new header `<format>` and should have no impact on existing code.

Proposed wording

Add a new entry `<format>` to table 18 in section 15.5.1.2 [headers] paragraph 2.

Change in table 35 of 16.3.1 [support.limits.general] paragraph 3:

Macro name	Value	Header(s)
[...]	[...]	[...]
__cpp_lib_filesystem	201703L	<filesystem>
__cpp_lib_format	201811L	<format>
__cpp_lib_gcd_lcm	201606L	<numeric>
[...]	[...]	[...]

Add a new section in 19 [utilities].

19.20 Formatting utilities [format]

19.20.1 Header `<format>` synopsis [format.syn]

```
namespace std {
  // [format.error], class format_error
  class format_error;

  // [format.formatter], formatter
  template<class charT> class basic_format_parse_context;
  using format_parse_context = basic_format_parse_context<char>;
  using wformat_parse_context = basic_format_parse_context<wchar_t>;
```

```
  template<class O, class charT> requires OutputIterator<O, const charT&>
    class basic_format_context;
  using format_context = basic_format_context<unspecified, char>;
  using wformat_context = basic_format_context<unspecified, wchar_t>;

  template<class T, class charT = char> struct formatter;

  // [format.arguments], arguments
  template<class Context> class basic_format_arg;

  template<class Visitor, class Context>
    see below visit_format_arg(Visitor&& vis, basic_format_arg<Context> arg);

  template<class Context, class... Args> struct format_arg_store; // exposition only

  template<class Context> class basic_format_args;
  using format_args = basic_format_args<format_context>;
  using wformat_args = basic_format_args<wformat_context>;

  template<class O, class charT>
    using format_args_t = basic_format_args<basic_format_context<O, charT>>;

  template<class Context = format_context, class... Args>
    format_arg_store<Context, Args...>
      make_format_args(const Args&... args);
  template<class... Args>
    format_arg_store<wformat_context, Args...>
      make_wformat_args(const Args&... args);

  // [format.functions], formatting functions
  template<class... Args>
    string format(string_view fmt, const Args&... args);
  template<class... Args>
    wstring format(wstring_view fmt, const Args&... args);

  string vformat(string_view fmt, format_args args);
  wstring vformat(wstring_view fmt, wformat_args args);

  template<OutputIterator<const char&> O, class... Args>
    O format_to(O out, string_view fmt, const Args&... args);
  template<OutputIterator<const wchar_t&> O, class... Args>
    O format_to(O out, wstring_view fmt, const Args&... args);

  template<OutputIterator<const char&> O>
    O vformat_to(O out, string_view fmt, format_args_t<O, char> args);
  template<OutputIterator<const wchar_t&> O>
    O vformat_to(O out, wstring_view fmt, format_args_t<O, wchar_t> args);

  template<class O>
    struct format_to_n_result {
      O out;
      iter_difference_t<O> size;
    };

  template<OutputIterator<const char&> O, class... Args>
    format_to_n_result<O> format_to_n(O out, iter_difference_t<O> n,
                                      string_view fmt, const Args&... args);
  template<OutputIterator<const wchar_t&> O, class... Args>
    format_to_n_result<O> format_to_n(O out, iter_difference_t<O> n,
                                      wstring_view fmt, const Args&... args);

  template<class... Args>
    size_t formatted_size(string_view fmt, const Args&... args);
  template<class... Args>
    size_t formatted_size(wstring_view fmt, const Args&... args);
}
```

19.20.2 Formatting functions [format.functions]

```
template<class... Args>
  string format(string_view fmt, const Args&... args);
```

Returns: `vformat(fmt, make_format_args(args...))`.

```
template<class... Args>
  wstring format(wstring_view fmt, const Args&... args);
```

Returns: `vformat(fmt, make_wformat_args(args...))`.

```
string vformat(string_view fmt, format_args args);
wstring vformat(wstring_view fmt, wformat_args args);
```

Returns: A string object holding the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt`.

Throws: `format_error` if `fmt` is not a valid format string.

```
template<OutputIterator<const char&> O, class... Args>
  O format_to(O out, string_view fmt, const Args&... args);
```

Returns: `vformat_to(out, fmt, make_format_args<basic_format_context<O, char>>(args...))`.

```
template<OutputIterator<const wchar_t&> O, class... Args>
  O format_to(O out, wstring_view fmt, const Args&... args);
```

Returns: `vformat_to(out, fmt, make_format_args<basic_format_context<O, wchar_t>>(args...))`.

```
template<OutputIterator<const char&> O>
  O vformat_to(O out, string_view fmt, format_args_t<O, char> args);
template<OutputIterator<const wchar_t&> O>
  O vformat_to(O out, wstring_view fmt, format_args_t<O, wchar_t> args);
```

Effects: Places the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt` into the range `[out, out + N)`, where `N` is the formatted output size.

Returns: `out + N`.

Throws: `format_error` if `fmt` is not a valid format string.

```
template<OutputIterator<const char&> O, class... Args>
  format_to_n_result<O> format_to_n(O out, iter_difference_t<O> n,
                                    string_view fmt, const Args&... args);
template<OutputIterator<const wchar_t&> O, class... Args>
  format_to_n_result<O> format_to_n(O out, iter_difference_t<O> n,
                                    wstring_view fmt, const Args&... args);
```

Let `N` be the formatted output size and `M` be `min(max(n, 0), N)`.

Effects: Places the character representation of formatting arguments provided by `args` formatted according to specifications given in `fmt` into the range `[out, out + M)`.

Returns: `{out + M, N}`.

Throws: `format_error` if `fmt` is not a valid format string.

```
template<class... Args>
  size_t formatted_size(string_view fmt, const Args&... args);
template<class... Args>
  size_t formatted_size(wstring_view fmt, const Args&... args);
```

Returns: The number of characters in the character representation of formatting arguments `args` formatted according to specifications given in `fmt`.

Throws: `format_error` if `fmt` is not a valid format string.

The `fmt` string consists of zero or more *replacement fields*, *escape sequences*, and ordinary multibyte characters. All ordinary multibyte characters are copied unchanged to the output. An escape sequence is one of `{{` or `}}`. It is replaced with `{` or `}` respectively in the output. The syntax of replacement fields is as follows:

```
  replacement-field ::= '{' [arg-id] [':' format-spec] '}'
  arg-id            ::= '0' | nonzero-digit [integer]
  integer           ::= digit [integer]
  nonzero-digit     ::= '1'...'9'
  digit             ::= '0'...'9'
```

The `arg-id` field specifies the index of the argument in `args` whose value is to be formatted and inserted into the output instead of the replacement field. The optional `format-spec` field specifies a non-default format for the replacement value.

[*Example:*

```
string s = format("{0}-{{", 8); // s == "8-{"
```

— *end example*]

If the numeric `arg-id`s in a format string are 0, 1, 2, ... in sequence, they can all be omitted (not just some) and the numbers 0, 1, 2, ... will be automatically used in that order. Mixing automatic and manual indexing is not allowed.

[*Example:*

```
string s0 = format("{} to {}",   "a", "b"); // OK: automatic indexing
string s1 = format("{1} to {0}", "a", "b"); // OK: manual indexing
string s2 = format("{0} to {}",  "a", "b"); // Error: mixing automatic and manual indexing
string s3 = format("{} to {1}",  "a", "b"); // Error: mixing automatic and manual indexing
```

— *end example*]

The `format-spec` field contains *format specifications* that define how the value should be presented, including such details as field width, alignment, padding, and decimal precision. Each type can define its own *formatting mini-language* or interpretation of the `format-spec` field. The syntax of format specifications is as follows:

```
format-spec     ::= std-format-spec | custom-format-spec
std-format-spec ::= [[fill] align] [sign] ['#'] ['0'] [width] ['.' precision] [type]
fill            ::= <a character other than '{' or '}'>
align           ::= '<' | '>' | '=' | '^'
sign            ::= '+' | '-' | ' '
width           ::= nonzero-digit [integer] | '{' arg-id '}'
precision       ::= integer | '{' arg-id '}'
type            ::= 'a' | 'A' | 'b' | 'B' | 'c' | 'd' | 'e' | 'E' | 'f' | 'F' |
                    'g' | 'G' | 'n' | 'o' | 'p' | 's' | 'x' | 'X'
```

where `std-format-spec` defines a common formatting mini-language supported by fundamental and string types, while `custom-format-spec` is a placeholder for user-defined mini-languages. Some of the formatting options are only supported by arithmetic types.

The `fill` character can be any character other than `'{'` or `'}'`. The presence of a fill character is signaled by the character following it, which must be one of the alignment options. If the second character of `format-spec` is not a valid alignment option, then it is assumed that both the fill character and the alignment option are absent.

Let `charT` be `decltype(fmt)::value_type`.

The meaning of the various alignment options is as follows:

Option	Meaning
`'<'`	Forces the field to be left-aligned within the available space. This is the default for non-arithmetic types, `charT`, and `bool`, unless an integer presentation type is specified.
`'>'`	Forces the field to be right-aligned within the available space. This is the default for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified.
`'='`	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form `+000000120`. This alignment option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified.
`'^'`	Forces the field to be centered within the available space by inserting `N / 2` and `N - N / 2` fill characters before and after the value respectively, where `N` is the total number of fill characters to insert.

[*Example:*

```
char c = 120;
string s0 = format("{:6}", 42);      // s0 == "    42"
string s1 = format("{:6}", 'x');     // s1 == "x     "
string s2 = format("{:*<6}", 'x');   // s2 == "x*****"
string s3 = format("{:*>6}", 'x');   // s3 == "*****x"
string s4 = format("{:*^6}", 'x');   // s4 == "**x***"
string s5 = format("{:=6}", 'x');    // Error: '=' with charT and no integer presentation type
string s6 = format("{:6d}", c);      // s6 == "   120"
string s7 = format("{:=+06d}", c);   // s7 == "+00120"
string s8 = format("{:6}", true);    // s8 == "true  "
```

— end example]

Note that unless a minimum field width is defined, the field width will be determined by the width of the content, meaning that the alignment option has no effect.

The `sign` option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. It can be one of the following:

Option	Meaning
`'+'`	Indicates that a sign should be used for both positive as well as negative numbers.
`'-'`	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used for positive numbers, and a minus sign for negative numbers.

The `sign` option applies to floating-point infinity and NaN.

[*Example:*

```
double inf = std::numeric_limits<double>::infinity();
double nan = std::numeric_limits<double>::quiet_NaN();
string s0 = format("{0:} {0:+} {0:-} {0: }", 1);   // s0 == "1 +1 1  1"
string s1 = format("{0:} {0:+} {0:-} {0: }", -1);  // s1 == "-1 -1 -1 -1"
string s2 = format("{0:} {0:+} {0:-} {0: }", inf); // s2 == "inf +inf inf  inf"
string s3 = format("{0:} {0:+} {0:-} {0: }", nan); // s3 == "nan +nan nan  nan"
```

— end example]

The `'#'` option causes the *alternate form* to be used for the conversion. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. For integers, when binary, octal, or hexadecimal output is used, this option adds the respective prefix `"0b"` (`"0B"`), `"0"`, or `"0x"` (`"0X"`) to the output value. Whether the prefix is lower-case or upper-case is determined by the case of the type format specifier. For floating-point numbers the alternate form causes the result of the conversion to always contain a decimal-point character, even if no digits follow it. Normally, a decimal-point character appears in the result of these conversions only if a digit follows it. In addition, for `'g'` and `'G'` conversions, trailing zeros are not removed from the result.

`width` is a decimal integer defining the minimum field width. If not specified, then the field width will be determined by the content.

Preceding the `width` field by a zero (`'0'`) character enables sign-aware zero-padding for arithmetic types. This is equivalent to a `fill` character of `'0'` with an `alignment` type of `'='`.

The `precision` field is a decimal integer defining the precision or maximum field size. It can only be used with floating-point and string types. For floating-point types this field specifies the formatting precision. For string types it specifies how many characters will be used from the string.

Finally, the `type` determines how the data should be presented.

The available string presentation types are:

Type	Meaning
`'s'`	Copies the string to the output.
none	The same as `'s'`.

The available `charT` presentation types are:

Type	Meaning
`'c'`	Copies the character to the output.
none	The same as `'c'`.

Formatting of objects of arithmetic types and `const void*` is done as if by calling `to_chars` and copying the output through the output iterator of the context with additional padding and adjustments as per format specifiers.

Let `[first, last)` be a range large enough to hold the `to_chars` output and `value` be the formatting argument value.

The available integer presentation types and their mapping to `to_chars` are:

Type	Meaning
'b'	to_chars(first, last, value, 2); using the '#' option with this type adds the prefix "0b" to the output.
'B'	The same as 'b', except that the '#' option adds the prefix "0B" to the output.
'd'	to_chars(first, last, value).
'o'	to_chars(first, last, value, 8); using the '#' option with this type adds the prefix "0" to the output.
'x'	to_chars(first, last, value, 16); using the '#' option with this type adds the prefix "0x" to the output.
'X'	The same as 'x', except that it uses uppercase letters for digits above 9 and the '#' option adds the prefix "0X" to the output.
'n'	The same as 'd', except that it uses the current global locale to insert the appropriate number separator characters.
none	The same as 'd' if formatting argument type is not charT or bool.

Integer presentation types can also be used with charT and bool values. Values of type bool are formatted using textual representation, either "true" or "false", if the presentation type is not specified.

[*Example:*

```
string s0 = format("{}", 42);                      // s0 == "42"
string s1 = format("{0:b} {0:d} {0:o} {0:x}", 42); // s1 == "101010 42 52 2a"
string s2 = format("{0:#x} {0:#X}", 42);           // s2 == "0x2a 0X2A"
string s3 = format("{:n}", 1234);                  // s3 == "1,234" (depends on the locale)
```

— *end example*]

The available floating-point presentation types and their mapping to to_chars are:

Type	Meaning
'a'	to_chars(first, last, value, chars_format::hex, precision) if precision is specified, to_chars(first, last, value, chars_format::hex) otherwise.
'A'	The same as 'a', except that it uses uppercase letters for digits above 9, "P" to indicate the exponent, "INF" for infinity, and "NAN" for NaN.
'e'	to_chars(first, last, value, chars_format::scientific, precision); precision defaults to 6 if not specified.
'E'	The same as 'e', except that it uses "E" to indicate exponent, "INF" for infinity, and "NAN" for NaN.
'f'	to_chars(first, last, value, chars_format::fixed, precision); precision defaults to 6 if not specified.
'F'	The same as 'f', except that it uses "INF" for infinity, and "NAN" for NaN.
'g'	to_chars(first, last, value, chars_format::general, precision); precision defaults to 6 if not specified.
'G'	The same as 'g', except that it uses "E" to indicate exponent, "INF" for infinity, and "NAN" for NaN.
'n'	The same as 'g', except that it uses the current global locale to insert the appropriate number separator characters.
none	to_chars(first, last, value, chars_format::general, precision) if precision is specified, to_chars(first, last, value) otherwise.

The available pointer presentation types and their mapping to to_chars are:

Type	Meaning
'p'	to_chars(first, last, reinterpret_cast<uintptr_t>(value), 16) with the prefix "0x" added to the output.
none	The same as 'p'.

A format string that doesn't conform to the current specification is invalid.

19.20.3 Formatter [format.formatter]

The functions defined in [format.functions] use specializations of the class template formatter to format individual arguments.

Each specialization of `formatter` is either enabled or disabled, as described below. [*Note:* Enabled specializations meet the *Formatter* requirements, and disabled specializations do not. — *end note*] Each header that declares the template `formatter` provides enabled specializations

- `template<> struct formatter<charT, charT>;`
- `template<> struct formatter<char, wchar_t>;`
- `template<> struct formatter<charT*, charT>;`
- `template<> struct formatter<const charT*, charT>;`
- `template<size_t N> struct formatter<const charT[N], charT>;`
- ```
  template<class traits, class Allocator>
    struct formatter<std::basic_string<charT, traits, Allocator>, charT>;
  ```
- ```
  template<class traits>
    struct formatter<std::basic_string_view<charT, traits>, charT>;
  ```
- specialiazations of `formatter` for `nullptr_t`, `void*`, `const void*`, `bool`, and all cv-unqualified standard integer types, extended integer types, and floating-point types as the first template argument and *charT* as the second template argument,

where *charT* is `char` or `wchar_t` (both sets of specializations are provided).

[*Note:* Specializations such as `formatter<wchar_t, char>` and `formatter<const char*, wchar_t>` that require implicit multibyte / wide string or character conversion are intentionally disabled. — *end note*]

For any types `T` and `charT` for which neither the library nor the user provides an explicit or partial specialization of the class template `formatter`, `formatter<T, charT>` is disabled.

If the library provides an explicit or partial specialization of `formatter<T, charT>`, that specialization is enabled except as noted otherwise.

If `F` is a disabled specialization of `formatter`, these values are `false`: `is_default_constructible_v<F>`, `is_copy_constructible_v<F>`, `is_move_constructible_v<F>`, `is_copy_assignable_v<F>`, `is_move_assignable_v<F>`.

An enabled specialization `formatter<T, charT>` will satisfy the *Formatter* requirements.

[*Example:*

```
#include <format>

enum color { red, green, blue };

const char* color_names[] = { "red", "green", "blue" };

template<> struct std::formatter<color> : std::formatter<const char*> {
  auto format(color c, format_context& ctx) {
    return formatter<const char*>::format(color_names[c], ctx);
  }
};

struct err {};

string s0 = std::format("{}", 42);     // OK: library-provided formatter
string s1 = std::format("{}", L"foo"); // Ill-formed: disabled formatter
string s2 = std::format("{}", red);    // OK: user-provided formatter
string s3 = std::format("{}", err{});  // Ill-formed: disabled formatter
```

— *end example*]

19.20.3.1 *Formatter* requirements [formatter.requirements]

A type `F` meets the *Formatter* requirements if:

— it satisfies the *Cpp17DefaultConstructible* (Table 24), *Cpp17CopyConstructible* (Table 26), *Cpp17CopyAssignable* (Table 28), and *Cpp17Destructible* (Table 29) requirements,

— it is swappable ([swappable.requirements]) for lvalues, and

— the expressions shown in Table 1 are valid and have the indicated semantics.

Given character type `charT`, output iterator type `O`, and formatting argument type `T`, in Table 1 `f` is a value of type `F`, `u` is an lvalue of type `T`, `t` is a value of a type convertible to (possibly const) `T`, `pc` is an lvalue of type `basic_format_parse_context<charT>` (denoted by `PC`), and `fc` is an lvalue of type `basic_format_context<O, charT>` (denoted by `FC`). `pc.begin()` points to the beginning of the `format-spec` ([format.syntax]) portion of the format string. If `format-spec` is empty then either `pc.begin() == pc.end()` or `*pc.begin() == '}'`.

Table 1 — *Formatter* requirements

Expression	Return type	Requirement
`f.parse(pc)`	`PC::iterator`	Shall parse `format-spec` for type `T`, store the parsed specifiers in `*this`, and return an iterator past the end of the parsed range.
`f.format(t, fc)`	`FC::iterator`	Shall format `t` according to the specifiers stored in `*this`, write the output to `fc.out()` and return an iterator past the end of the output range.
`f.format(u, fc)`	`FC::iterator`	Shall not modify u.

19.20.3.2 Class template `basic_format_parse_context` [format.parse_context]

```
namespace std {
  template<class charT>
  class basic_format_parse_context {
  public:
    using char_type = charT;
    using const_iterator = typename basic_string_view<charT>::const_iterator;
    using iterator = const_iterator;

  private:
    iterator begin_;                              // exposition only
    iterator end_;                                // exposition only
    enum indexing { unknown, manual, automatic }; // exposition only
    indexing indexing_;                           // exposition only
    size_t next_arg_id_;                          // exposition only

  public:
    explicit constexpr basic_format_parse_context(basic_string_view<charT> fmt) noexcept;
    basic_format_parse_context(const basic_format_parse_context&) = delete;
    basic_format_parse_context& operator=(const basic_format_parse_context&) = delete;

    constexpr const_iterator begin() const noexcept;
    constexpr const_iterator end() const noexcept;
    constexpr void advance_to(iterator it);

    constexpr size_t next_arg_id();
    constexpr void check_arg_id(size_t id);
  };
}
```

An instance of `basic_format_parse_context` holds the format string parsing state consisting of the format string range being parsed and the argument counter for automatic indexing.

```
explicit constexpr basic_format_parse_context(basic_string_view<charT> fmt) noexcept;
```

Effects: Initializes `begin_` with `fmt.begin()`, `end_` with `fmt.end()`, `indexing_` with `unknown`, and `next_arg_id_` with `0`.

```
constexpr const_iterator begin() const noexcept;
```

Returns: `begin_`.

```
constexpr const_iterator end() const noexcept;
```

Returns: `end_`.

```
constexpr void advance_to(iterator it);
```

Requires: `end()` shall be reachable from `it`.

Effects: Equivalent to: `begin_ = it;`

```
constexpr size_t next_arg_id();
```

Effects: Equivalent to:

```
if (indexing_ == unknown)
  indexing_ = automatic;
return next_arg_id_++;
```

Throws: `format_error` if `indexing_ == manual` which indicates mixing of automatic and manual argument indexing.

```
constexpr void check_arg_id(size_t id);
```

Effects: Equivalent to:

```
if (indexing_ == unknown)
  indexing_ = manual;
```

Throws: `format_error` if `indexing_ == automatic` which indicates mixing of automatic and manual argument indexing.

19.20.3.3 Class template `basic_format_context` [format.context]

```
namespace std {
  template<class O, class charT> requires OutputIterator<O, const charT&>
  class basic_format_context {
    basic_format_parse_context<charT> parse_context_; // exposition only
    basic_format_args<basic_format_context> args_;    // exposition only
    O out_;                                           // exposition only

  public:
    using iterator = O;
    using char_type = charT;

    template<class T>
      using formatter_type = formatter<T>;

    basic_format_parse_context<charT>& parse_context() noexcept;
    basic_format_arg<basic_format_context> arg(size_t id) const;

    iterator out();
    void advance_to(iterator it);
  };
}
```

An instance of `basic_format_context` holds formatting state consisting of the format string parsing context, formatting arguments, and output iterator.

```
basic_format_parse_context<charT>& parse_context() noexcept;
```

Returns: `parse_context_`.

```
basic_format_arg<basic_format_context> arg(size_t id) const;
```

Returns: `args_.get(id)`.

```
iterator out();
```

Returns: `out_`.

```
void advance_to(iterator it);
```

Effects: Equivalent to: `out_ = it;`

[*Example:*

```
  struct S {
    int value;
  };

  template<> struct std::formatter<S> {
    int width_arg_id = 0;

    // Parses a width argument id in the format { <digit> }.
    constexpr auto parse(format_parse_context& ctx) {
      auto iter = ctx.begin();
      auto get_char = [&]() { return iter != ctx.end() ? *iter : 0; };
```

```
      if (get_char() != '{')
        return iter;
      ++iter;
      char c = get_char();
      if (!std::isdigit(c) || (++iter, get_char()) != '}')
        throw format_error("invalid format");
      width_arg_id = c - '0';
      ctx.check_arg_id(width_arg_id);
      return ++iter;
    }

    // Formats S with width given by the argument width_arg_id.
    auto format(S s, format_context& ctx) {
      int width = visit_format_arg([](auto value) -> int {
        if constexpr (!std::is_integral_v<decltype(value)>)
          throw format_error("width is not integral");
        else if (value < 0 || value > std::numeric_limits<int>::max())
          throw format_error("invalid width");
        else
          return value;
        }, ctx.arg(width_arg_id));
      return format_to(ctx.out(), "{0:{1}}", s.value, width);
    }
  };

  std::string s = format("{0:{1}}", S{42}, 10); // s == "        42"
```

— *end example*]

19.20.4 Arguments [format.arguments]

19.20.4.1 Class template `basic_format_arg` [format.arg]

```
namespace std {
  template<class Context>
  class basic_format_arg {
  public:
    class handle;

    using char_type = typename Context::char_type;                      // exposition only

    variant<monostate, bool, char_type,
            int, unsigned int, long long int, unsigned long long int,
            double, long double,
            const char_type*, basic_string_view<char_type>,
            const void*, handle> value;                                 // exposition only

    basic_format_arg() noexcept;

    template<Integral I> explicit basic_format_arg(I val) noexcept;     // exposition only
    explicit basic_format_arg(float val) noexcept;                      // exposition only
    explicit basic_format_arg(double val) noexcept;                     // exposition only
    explicit basic_format_arg(long double val) noexcept;                // exposition only
    explicit basic_format_arg(const char_type* val) noexcept;           // exposition only
    explicit basic_format_arg(const void* val) noexcept;                // exposition only
    explicit basic_format_arg(nullptr_t) noexcept;                      // exposition only

    template<class traits>
      explicit basic_format_arg(
        basic_string_view<char_type, traits> val) noexcept;             // exposition only

    template<class traits, class Allocator>
      explicit basic_format_arg(
        const basic_string<char_type, traits, Allocator>& val) noexcept; // exposition only

    template<class T>
      explicit basic_format_arg(const T& val) noexcept;                 // exposition only

    explicit operator bool() const noexcept;

    bool is_arithmetic() const noexcept;
    bool is_integral() const noexcept;
  };
}
```

An instance of `basic_format_arg` provides access to a formatting argument for user-defined formatters.

```
basic_format_arg() noexcept;
```

Ensures: `!(*this)`.

```
template<Integral I> explicit basic_format_arg(I val) noexcept;
```

Effects:

- if `I` is `bool` or `char_type`, initializes `value` with `val`,
- if `I` is `char` and `char_type` is `wchar_t`, initializes `value` with `static_cast<wchar_t>(val)`,
- if `I` is a standard signed integer type or an extended signed integer type and `sizeof(I) <= sizeof(int)`, initializes `value` with `static_cast<int>(val)`,
- if `I` is a standard unsigned integer type or an extended unsigned integer type and `sizeof(I) <= sizeof(unsigned int)`, initializes `value` with `static_cast<unsigned int>(val)`,
- if `I` is a standard signed integer type or an extended signed integer type and `sizeof(I) <= sizeof(long long int)`, initializes `value` with `static_cast<long long int>(val)`,
- if `I` is a standard unsigned integer type or an extended unsigned integer type and `sizeof(I) <= sizeof(unsigned long long int)`, initializes `value` with `static_cast<unsigned long long int>(val)`,
- otherwise the program is ill-formed.

```
explicit basic_format_arg(float val) noexcept;
```

Effects: Initializes `value` with `static_cast<double>(val)`.

```
explicit basic_format_arg(double val) noexcept;
explicit basic_format_arg(long double val) noexcept;
explicit basic_format_arg(const char_type* val) noexcept;
explicit basic_format_arg(const void* val) noexcept;
```

Effects: Initializes `value` with `val`.

```
explicit basic_format_arg(nullptr_t) noexcept;
```

Effects: Initializes `value` with `static_cast<const void*>(nullptr)`.

```
template<class traits>
  explicit basic_format_arg(basic_string_view<char_type, traits> val) noexcept;
```

Effects: Initializes `value` with `basic_string_view<char_type>(val.data(), val.size())`.

```
template<class traits, class Allocator>
  explicit basic_format_arg(
    const basic_string<char_type, traits, Allocator>& val) noexcept;
```

Effects: Initializes `value` with `basic_string_view<char_type>(val.data(), val.size())`.

```
template<class T> explicit basic_format_arg(const T& val) noexcept;
```

Effects: Initializes `value` with `handle(val)`.

Remarks: This constructor shall not participate in overload resolution unless `formatter<T, char_type>` is enabled.

```
explicit operator bool() const noexcept;
```

Returns: `!holds_alternative<monostate>(value)`.

```
bool is_arithmetic() const noexcept;
```

Returns: `visit([](auto v) { return is_arithmetic_v<decltype(v)>; }, value)`.

```
bool is_integral() const noexcept;
```

Returns: `visit([](auto v) { return is_integral_v<decltype(v)>; }, value)`.

[*Note:* Implementations are encouraged to take advantage of the ordering of the alternative types in `value` and implement `is_arithmetic` and `is_integral` more efficiently as a range check on `value.index()`. — *end note*]

```
namespace std {
  template<class Context>
  class basic_format_arg<Context>::handle {
    const void* ptr_;                                        // exposition only
    void (*format_)(Context&, const void*);                  // exposition only

  public:
    template<class T> explicit handle(const T& val) noexcept; // exposition only

    void format(Context& ctx) const;
  };
}
```

The class `handle` allows formatting an object of a non-fundamental type.

```
template<class T> explicit handle(const T& val) noexcept;
```

Effects: Initializes `ptr_` with `&val` and `format_` with

```
[](Context& ctx, const void* ptr) {
    typename Context::template formatter_type<T> f;
    ctx.parse_context().advance_to(f.parse(ctx.parse_context()));
    ctx.advance_to(f.format(*static_cast<const T*>(ptr), ctx));
  }
```

```
void format(Context& ctx) const;
```

Effects: Equivalent to: `format_(ctx, ptr_);`

19.20.4.2 Argument visitation [format.visit]

```
template<class Visitor, class Context>
  see below visit_format_arg(Visitor&& vis, basic_format_arg<Context> arg);
```

Returns: `visit(vis, arg.value)`.

Remarks: The return type is the type of the expression in the *Returns* section.

19.20.4.3 Class template `format_arg_store`

```
namespace std {
  template<class Context, class... Args>
  struct format_arg_store { // exposition only
    array<basic_format_arg<Context>, sizeof...(Args)> args;
  };
}
```

19.20.4.4 Class template `basic_format_args`

```
namespace std {
  template<class Context>
  class basic_format_args {
    size_t size_;                            // exposition only
    const basic_format_arg<Context>* data_; // exposition only

  public:
    basic_format_args() noexcept;

    template<class... Args>
      basic_format_args(const format_arg_store<Context, Args...>& store) noexcept;

    basic_format_arg<Context> get(size_t i) const noexcept;
  };
}
```

An instance `basic_format_args` provides access to formatting arguments.

```
basic_format_args() noexcept;
```

Effects: Initializes `size_` with `0`.

```
template<class... Args>
  basic_format_args(const format_arg_store<Context, Args...>& store) noexcept;
```

Effects: Initializes `size_` with `sizeof...(Args)` and `data_` with `store.args.data()`.

```
basic_format_arg<Context> get(size_t i) const noexcept;
```

Returns: `i < size_ ? data_[i] : basic_format_arg<Context>()`.

[*Note:* Implementations are encouraged to optimize the representation of `basic_format_args` for small number of formatting arguments by storing indices of type alternatives separately from values and packing the former. — *end note*]

19.20.4.5 Function template `make_format_args`

```
template<class Context = format_context, class... Args>
  format_arg_store<Context, Args...> make_format_args(const Args&... args);
```

Returns: `{basic_format_arg<Context>(args)...}`.

19.20.4.6 Function template `make_wformat_args`

```
template<class... Args>
  format_arg_store<wformat_context, Args...> make_wformat_args(const Args&... args);
```

Returns: `{basic_format_arg<wformat_context>(args)...}`.

19.20.5 Class `format_error` [format.error]

```
namespace std {
  class format_error : public runtime_error {
  public:
    explicit format_error(const string& what_arg);
    explicit format_error(const char* what_arg);
  };
}
```

The class `format_error` defines the type of objects thrown as exceptions to report errors from the formatting library.

```
format_error(const string& what_arg);
```

Ensures: `strcmp(what(), what_arg.c_str()) == 0`.

```
format_error(const char* what_arg);
```

Ensures: `strcmp(what(), what_arg) == 0`.

Related work

The Boost Format library [8] is an established formatting library that uses `printf`-like format string syntax with extensions. The main differences between this library and the current proposal are:

- Syntax: for the reasons described in section Format String Syntax this proposal uses a new syntax instead of extending the `printf` one. This allows much simpler and easier to parse grammar, not burdened by legacy specifiers used to convey type information. For example, Boost Format has two ways to refer to an argument by index and allows but ignores some format specifiers.
- API: Boost Format uses `operator%` to pass formatting arguments while this proposal uses variadic function templates.
- Performance: the implementation of this proposal is several times faster that the implementation of Boost Format on tinyformat benchmarks [9], generates smaller binary code and is faster to compile.

A `printf`-like Interface for the Streams Library [10] is similar to the Boost Format library but uses variadic templates instead of `operator%`. Unfortunately it hasn't been updated since 2013 and the same arguments about format string syntax apply to it.

The FastFormat library [11] is another well-known formatting library. Similarly to this proposal, FastFormat uses brace-delimited format specifiers, but otherwise the format string syntax is different and the library has significant limitations [12]:

> Three features that have no hope of being accommodated within the current design are:
>
> - Leading zeros (or any other non-space padding)
> - Octal/hexadecimal encoding

- Runtime width/alignment specification

Formatting facilities of the Folly library [13] are the closest to the current proposal. Folly also uses Python-like format string syntax nearly identical to the one described here. However, the API details are quite different. The current proposal tries to address performance and code bloat issues that are largely ignored by Folly Format. For instance formatting functions in Folly Format are parameterized on all argument types while in this proposal, only the inlined wrapper functions are, which results in much smaller binary code and better compile times.

Implementation

An implementation of this proposal is available in the `std` branch of the open-source fmt library [14].

Acknowledgements

Thanks to Antony Polukhin, Beman Dawes, Bengt Gustafsson, Eric Niebler, Jason McKesson, Jeffrey Yasskin, Joël Lamotte, Lars Gullik Bjønnes, Lee Howes, Louis Dionne, Matt Clarke, Michael Park, Sergey Ignatchenko, Thiago Macieira, Zach Laine, Zhihao Yuan and participants of the Library Evolution Working Group for their feedback, support, constructive criticism and contributions to the proposal. Special thanks to Howard Hinnant who encouraged me to write the proposal and gave useful early advice on how to go about it.

The format string syntax is based on the Python documentation [3].

References

[1] *The `fprintf` function. ISO/IEC 9899:2011. 7.21.6.1.*
[2] *fprintf, printf, snprintf, sprintf - print formatted output. The Open Group Base Specifications Issue 6 IEEE Std 1003.1, 2004 Edition.*
[3] *6.1.3. Format String Syntax. Python 3.5.2 documentation.*
[4] *String.Format Method. .NET Framework Class Library.*
[5] *Module `std::fmt`. The Rust Standard Library.*
[6] *Format Specification Syntax: printf and wprintf Functions. C++ Language and Standard Libraries.*
[7] *10.4.2 Rearranging printf Arguments. The GNU Awk User's Guide.*
[8] *Boost Format library. Boost 1.63 documentation.*
[9] *Speed Test. The fmt library repository.*
[10] *A printf-like Interface for the Streams Library (revision 1).*
[11] *The FastFormat library website.*
[12] *An Introduction to Fast Format (Part 1): The State of the Art. Overload Journal #89 - February 2009*
[13] *The folly library repository.*
[14] *The fmt library repository.*
[15] *P0424: Reconsidering literal operator templates for strings.*
[16] *D0355: Extending <chrono> to Calendars and Time Zones.*
[17] *P0067: Elementary string conversions.*
[18] *MPark.Format: Compile-time Checked, Type-Safe Formatting in C++14.*
[19] *Google Benchmark: A microbenchmark support library.*
[20] *N4412: Shortcomings of iostreams.*

Appendix A: Benchmarks

To demonstrate that the formatting functions described in this paper can be implemented efficiently, we compare the reference implementation [14] of `format` and `format_to` to `sprintf`, `ostringstream` and `to_string` on the following benchmark. This benchmark generates a set of integers with random numbers of digits, applies each method to convert each integer into a string (either `std::string` or a char buffer depending on the API) and uses the Google Benchmark library [19] to measure timings:

```
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <limits>
#include <sstream>
#include <string>
#include <utility>
```

```
#include <vector>

#include <benchmark/benchmark.h>
#include <fmt/format.h>

// Returns a pair with the smallest and the largest value of integral type T
// with the given number of digits.
template<typename T>
std::pair<T, T> range(int num_digits) {
  T first = std::pow(T(10), num_digits - 1);
  int max_digits = std::numeric_limits<T>::digits10 + 1;
  T last = num_digits < max_digits ? first * 10 - 1 :
                                     std::numeric_limits<T>::max();
  return {num_digits > 1 ? first : 0, last};
}

// Generates values of integral type T with random number of digits.
template<typename T>
std::vector<T> generate_random_data(int numbers_per_digit) {
  int max_digits = std::numeric_limits<T>::digits10 + 1;
  std::vector<T> data;
  data.reserve(max_digits * numbers_per_digit);
  for (int i = 1; i <= max_digits; ++i) {
    auto r = range<T>(i);
    auto value = r.first;
    std::generate_n(std::back_inserter(data), numbers_per_digit, [=]() mutable {
      T result = value;
      value = value < r.second ? value + 1 : r.first;
      return result;
    });
  }
  std::random_shuffle(data.begin(), data.end());
  return data;
}

auto data = generate_random_data<int>(1000);

void sprintf(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      char buffer[12];
      result += std::sprintf(buffer, "%d", i);
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(sprintf);

void ostringstream(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      std::ostringstream ss;
      ss << i;
      result += ss.str().size();
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(ostringstream);

void to_string(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data)
      result += std::to_string(i).size();
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(to_string);

void format(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data)
      result += fmt::format("{}", i).size();
  }
  benchmark::DoNotOptimize(result);
```

```
}
BENCHMARK(format);

void format_to(benchmark::State &s) {
  size_t result = 0;
  while (s.KeepRunning()) {
    for (auto i: data) {
      char buffer[12];
      result += fmt::format_to(buffer, "{}", i) - buffer;
    }
  }
  benchmark::DoNotOptimize(result);
}
BENCHMARK(format_to);

BENCHMARK_MAIN();
```

The benchmark was compiled with clang (Apple LLVM version 9.0.0 clang-900.0.39.2) with `-O3 -DNDEBUG` and run on a macOS system. Below are the results:

```
Run on (4 X 3100 MHz CPU s)
2018-01-27 07:12:00
Benchmark              Time           CPU Iterations
----------------------------------------------------
sprintf           882311 ns     881076 ns        781
ostringstream    2892035 ns    2888975 ns        242
to_string        1167422 ns    1166831 ns        610
format            675636 ns     674382 ns       1045
format_to         499376 ns     498996 ns       1263
```

The `format` and `format_to` functions show much better performance than the other methods. The `format` function that constructs `std::string` is even 30% faster than the system's version of `sprintf` that uses stack-allocated `char` buffer. `format_to` with a stack-allocated buffer is ~60% faster than `sprintf`.

Appendix B: Binary code comparison

In this section we compare per-call binary code size between the reference implementation that uses techniques described in section Binary footprint and standard formatting facilities. All the code snippets are compiled with clang (Apple LLVM version 9.0.0 clang-900.0.39.2) with `-O3 -DNDEBUG -c -std=c++14` and the resulted binaries are disassembled with `objdump -S`:

```
void consume(const char*);

void sprintf_test() {
  char buffer[100];
  sprintf(buffer, "The answer is %d.", 42);
  consume(buffer);
}

__Z12sprintf_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       53      pushq   %rbx
       5:       48 83 ec 78     subq    $120, %rsp
       9:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      10:       48 8b 00        movq    (%rax), %rax
      13:       48 89 45 f0     movq    %rax, -16(%rbp)
      17:       48 8d 35 37 00 00 00    leaq    55(%rip), %rsi
      1e:       48 8d 5d 80     leaq    -128(%rbp), %rbx
      22:       ba 2a 00 00 00  movl    $42, %edx
      27:       31 c0   xorl    %eax, %eax
      29:       48 89 df        movq    %rbx, %rdi
      2c:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x31>
      31:       48 89 df        movq    %rbx, %rdi
      34:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x39>
      39:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      40:       48 8b 00        movq    (%rax), %rax
      43:       48 3b 45 f0     cmpq    -16(%rbp), %rax
      47:       75 07   jne     7 <__Z12sprintf_testv+0x50>
      49:       48 83 c4 78     addq    $120, %rsp
      4d:       5b      popq    %rbx
      4e:       5d      popq    %rbp
      4f:       c3      retq
      50:       e8 00 00 00 00  callq   0 <__Z12sprintf_testv+0x55>

void format_test() {
```

```
  consume(format("The answer is {}.", 42).c_str());
}

__Z11format_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       53      pushq   %rbx
       5:       48 83 ec 28     subq    $40, %rsp
       9:       48 c7 45 d0 2a 00 00 00         movq    $42, -48(%rbp)
      11:       48 8d 35 f4 83 01 00    leaq    99316(%rip), %rsi
      18:       48 8d 7d e0     leaq    -32(%rbp), %rdi
      1c:       4c 8d 45 d0     leaq    -48(%rbp), %r8
      20:       ba 11 00 00 00  movl    $17, %edx
      25:       b9 02 00 00 00  movl    $2, %ecx
      2a:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x2F>
      2f:       f6 45 e0 01     testb   $1, -32(%rbp)
      33:       48 8d 7d e1     leaq    -31(%rbp), %rdi
      37:       48 0f 45 7d f0  cmovneq -16(%rbp), %rdi
      3c:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x41>
      41:       f6 45 e0 01     testb   $1, -32(%rbp)
      45:       74 09   je      9 <__Z11format_testv+0x50>
      47:       48 8b 7d f0     movq    -16(%rbp), %rdi
      4b:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x50>
      50:       48 83 c4 28     addq    $40, %rsp
      54:       5b      popq    %rbx
      55:       5d      popq    %rbp
      56:       c3      retq
      57:       48 89 c3        movq    %rax, %rbx
      5a:       f6 45 e0 01     testb   $1, -32(%rbp)
      5e:       74 09   je      9 <__Z11format_testv+0x69>
      60:       48 8b 7d f0     movq    -16(%rbp), %rdi
      64:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x69>
      69:       48 89 df        movq    %rbx, %rdi
      6c:       e8 00 00 00 00  callq   0 <__Z11format_testv+0x71>
      71:       66 66 66 66 66 66 2e 0f 1f 84 00 00 00 00 00    nopw    %cs:(%rax,%rax)

void ostringstream_test() {
  std::ostringstream ss;
  ss << "The answer is " << 42 << ".";
  consume(ss.str().c_str());
}

__Z18ostringstream_testv:
       0:     55      pushq   %rbp
       1:     48 89 e5        movq    %rsp, %rbp
       4:     41 57   pushq   %r15
       6:     41 56   pushq   %r14
       8:     41 55   pushq   %r13
       a:     41 54   pushq   %r12
       c:     53      pushq   %rbx
       d:     48 81 ec 38 01 00 00    subq    $312, %rsp
      14:     4c 8d b5 18 ff ff ff    leaq    -232(%rbp), %r14
      1b:     4c 8d a5 b0 fe ff ff    leaq    -336(%rbp), %r12
      22:     48 8b 05 00 00 00 00    movq    (%rip), %rax
      29:     48 8d 48 18     leaq    24(%rax), %rcx
      2d:     48 89 8d a8 fe ff ff    movq    %rcx, -344(%rbp)
      34:     48 83 c0 40     addq    $64, %rax
      38:     48 89 85 18 ff ff ff    movq    %rax, -232(%rbp)
      3f:     4c 89 f7        movq    %r14, %rdi
      42:     4c 89 e6        movq    %r12, %rsi
      45:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x4A>
      4a:     48 c7 45 a0 00 00 00 00         movq    $0, -96(%rbp)
      52:     c7 45 a8 ff ff ff ff    movl    $4294967295, -88(%rbp)
      59:     48 8b 1d 00 00 00 00    movq    (%rip), %rbx
      60:     4c 8d 6b 18     leaq    24(%rbx), %r13
      64:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
      6b:     48 83 c3 40     addq    $64, %rbx
      6f:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
      76:     4c 89 e7        movq    %r12, %rdi
      79:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x7E>
      7e:     4c 8b 3d 00 00 00 00    movq    (%rip), %r15
      85:     49 83 c7 10     addq    $16, %r15
      89:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
      90:     48 c7 85 08 ff ff ff 00 00 00 00        movq    $0, -248(%rbp)
      9b:     48 c7 85 00 ff ff ff 00 00 00 00        movq    $0, -256(%rbp)
      a6:     48 c7 85 f8 fe ff ff 00 00 00 00        movq    $0, -264(%rbp)
      b1:     48 c7 85 f0 fe ff ff 00 00 00 00        movq    $0, -272(%rbp)
      bc:     c7 85 10 ff ff ff 10 00 00 00   movl    $16, -240(%rbp)
      c6:     0f 57 c0        xorps   %xmm0, %xmm0
```

```
 c9:     0f 29 45 b0     movaps  %xmm0, -80(%rbp)
 cd:     48 c7 45 c0 00 00 00 00         movq    $0, -64(%rbp)
 d5:     48 8d 75 b0     leaq    -80(%rbp), %rsi
 d9:     4c 89 e7        movq    %r12, %rdi
 dc:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0xE1>
 e1:     f6 45 b0 01     testb   $1, -80(%rbp)
 e5:     74 09   je      9 <__Z18ostringstream_testv+0xF0>
 e7:     48 8b 7d c0     movq    -64(%rbp), %rdi
 eb:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0xF0>
 f0:     48 8d 35 dd 10 00 00    leaq    4317(%rip), %rsi
 f7:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
 fe:     ba 0e 00 00 00  movl    $14, %edx
103:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x108>
108:     be 2a 00 00 00  movl    $42, %esi
10d:     48 89 c7        movq    %rax, %rdi
110:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x115>
115:     48 8d 35 c7 10 00 00    leaq    4295(%rip), %rsi
11c:     ba 01 00 00 00  movl    $1, %edx
121:     48 89 c7        movq    %rax, %rdi
124:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x129>
129:     48 8d 7d b0     leaq    -80(%rbp), %rdi
12d:     4c 89 e6        movq    %r12, %rsi
130:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x135>
135:     f6 45 b0 01     testb   $1, -80(%rbp)
139:     48 8d 7d b1     leaq    -79(%rbp), %rdi
13d:     48 0f 45 7d c0  cmovneq -64(%rbp), %rdi
142:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x147>
147:     f6 45 b0 01     testb   $1, -80(%rbp)
14b:     74 09   je      9 <__Z18ostringstream_testv+0x156>
14d:     48 8b 7d c0     movq    -64(%rbp), %rdi
151:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x156>
156:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
15d:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
164:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
16b:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
172:     74 0c   je      12 <__Z18ostringstream_testv+0x180>
174:     48 8b bd 00 ff ff ff    movq    -256(%rbp), %rdi
17b:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x180>
180:     4c 89 e7        movq    %r12, %rdi
183:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x188>
188:     48 8b 35 00 00 00 00    movq    (%rip), %rsi
18f:     48 83 c6 08     addq    $8, %rsi
193:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
19a:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x19F>
19f:     4c 89 f7        movq    %r14, %rdi
1a2:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1A7>
1a7:     48 81 c4 38 01 00 00    addq    $312, %rsp
1ae:     5b      popq    %rbx
1af:     41 5c   popq    %r12
1b1:     41 5d   popq    %r13
1b3:     41 5e   popq    %r14
1b5:     41 5f   popq    %r15
1b7:     5d      popq    %rbp
1b8:     c3      retq
1b9:     48 89 45 d0     movq    %rax, -48(%rbp)
1bd:     f6 45 b0 01     testb   $1, -80(%rbp)
1c1:     74 3b   je      59 <__Z18ostringstream_testv+0x1FE>
1c3:     48 8b 7d c0     movq    -64(%rbp), %rdi
1c7:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1CC>
1cc:     eb 30   jmp     48 <__Z18ostringstream_testv+0x1FE>
1ce:     eb 2a   jmp     42 <__Z18ostringstream_testv+0x1FA>
1d0:     48 89 45 d0     movq    %rax, -48(%rbp)
1d4:     f6 45 b0 01     testb   $1, -80(%rbp)
1d8:     74 39   je      57 <__Z18ostringstream_testv+0x213>
1da:     48 8b 7d c0     movq    -64(%rbp), %rdi
1de:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x1E3>
1e3:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
1ea:     75 30   jne     48 <__Z18ostringstream_testv+0x21C>
1ec:     eb 3a   jmp     58 <__Z18ostringstream_testv+0x228>
1ee:     48 89 45 d0     movq    %rax, -48(%rbp)
1f2:     eb 3c   jmp     60 <__Z18ostringstream_testv+0x230>
1f4:     48 89 45 d0     movq    %rax, -48(%rbp)
1f8:     eb 4d   jmp     77 <__Z18ostringstream_testv+0x247>
1fa:     48 89 45 d0     movq    %rax, -48(%rbp)
1fe:     4c 89 ad a8 fe ff ff    movq    %r13, -344(%rbp)
205:     48 89 9d 18 ff ff ff    movq    %rbx, -232(%rbp)
20c:     4c 89 bd b0 fe ff ff    movq    %r15, -336(%rbp)
213:     f6 85 f0 fe ff ff 01    testb   $1, -272(%rbp)
21a:     74 0c   je      12 <__Z18ostringstream_testv+0x228>
```

```
      21c:     48 8b bd 00 ff ff ff    movq    -256(%rbp), %rdi
      223:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x228>
      228:     4c 89 e7        movq    %r12, %rdi
      22b:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x230>
      230:     48 8b 35 00 00 00 00    movq    (%rip), %rsi
      237:     48 83 c6 08     addq    $8, %rsi
      23b:     48 8d bd a8 fe ff ff    leaq    -344(%rbp), %rdi
      242:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x247>
      247:     4c 89 f7        movq    %r14, %rdi
      24a:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x24F>
      24f:     48 8b 7d d0     movq    -48(%rbp), %rdi
      253:     e8 00 00 00 00  callq   0 <__Z18ostringstream_testv+0x258>
      258:     0f 1f 84 00 00 00 00 00         nopl    (%rax,%rax)
```

The code generated for the `format_test` function that uses the reference implementation of the `format` function described in this proposal is several times smaller than the `ostringstream` code and only 40% larger than the one generated for `sprintf` which is a moderate price to pay for full type and memory safety.

The following factors contribute to the difference in binary code size between `format` and `sprintf`:

- Passing format string as `string_view` instead of `const char*`.
- Using `string` instead of a `char` buffer.
- Preparing the array of formatting arguments.

We can exclude the first two factors from the experiment by mimicking parts of the `sprintf` API:

```
int vraw_format(char* buffer, const char* format, format_args args);

template<typename... Args>
inline int raw_format(char* buffer, const char* format, const Args&... args) {
  return vraw_format(buffer, format, make_format_args(args...));
}

void raw_format_test() {
  char buffer[100];
  raw_format(buffer, "The answer is {}.", 42);
}

__Z15raw_format_testv:
       0:       55      pushq   %rbp
       1:       48 89 e5        movq    %rsp, %rbp
       4:       48 81 ec 80 00 00 00    subq    $128, %rsp
       b:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      12:       48 8b 00        movq    (%rax), %rax
      15:       48 89 45 f8     movq    %rax, -8(%rbp)
      19:       48 c7 45 80 2a 00 00 00         movq    $42, -128(%rbp)
      21:       48 8d 35 24 12 00 00    leaq    4644(%rip), %rsi
      28:       48 8d 7d 90     leaq    -112(%rbp), %rdi
      2c:       48 8d 4d 80     leaq    -128(%rbp), %rcx
      30:       ba 02 00 00 00  movl    $2, %edx
      35:       e8 00 00 00 00  callq   0 <__Z15raw_format_testv+0x3A>
      3a:       48 8b 05 00 00 00 00    movq    (%rip), %rax
      41:       48 8b 00        movq    (%rax), %rax
      44:       48 3b 45 f8     cmpq    -8(%rbp), %rax
      48:       75 09   jne     9 <__Z15raw_format_testv+0x53>
      4a:       48 81 c4 80 00 00 00    addq    $128, %rsp
      51:       5d      popq    %rbp
      52:       c3      retq
      53:       e8 00 00 00 00  callq   0 <__Z15raw_format_testv+0x58>
      58:       0f 1f 84 00 00 00 00 00         nopl    (%rax,%rax)
```

This shows that passing formatting arguments adds very little overhead and is comparable with `sprintf`.